

Smarter 100mV-Durchgangsprüfer

Warum eine Prüfspannung von 0,1V eine gute Idee sein kann

Aus der Praxis entstanden

Einen Durchgangsprüfer nutzt jeder Elektroniker, Maker, Hobbyist, Profi oder Entwicklungsingenieur. Von der altehrwürdigen „Klingelbox“ bis zum selbstgebauten „Piepser“ gibt es zahllose Varianten. Fast jedes Multimeter bietet eine Möglichkeit zum „continuity test“.

Und doch gibt es noch Verbesserungen, z.B. die Durchgangsprüfung mit niedriger Spannung. Die 100mV-Prüfspannung überwindet keinen Halbleiterübergang, nicht einmal den einer Schottkydiode. Und er vermeidet viele andere Probleme höherer Prüfspannungen.

Auf diese Idee brachte mich ein Beitrag von Jez Siddons in ELEKTOR 11/12-25 [1]. Jez wählte den Weg eines „self educational projects“ mit dem Ziel, eine Menge über den Aufbau von Operationsverstärkern zu lernen und zu vermitteln. Das führt zu einem diskreten Aufbau der Schaltung mit 43 Widerständen, 12 Dioden und 19 Transistoren. Ein schönes und lehrreiches Projekt.

Auf Basis seiner schönen Idee wurde eine MCU-basierte Lösung mit wenigen Komponenten und zusätzlichen Leistungsmerkmalen entwickelt. Diese Schaltung (es gibt eine Platine dazu) beschreibe ich hier. Der Aufbau erfolgt mit SMD-Bausteinen, die man von Hand löten kann.

Da das Gerät ausgesprochen kompakt ausfällt, werden für den Prototypen 2mm Steckverbindungen und passende Prüfklemmen verwendet.

Designziele

Tests an aufgebauten Platinen

Wichtigstes Designziel ist die Testmöglichkeit von Verbindungen in eingebautem Zustand. Weil Transistor- und Diodenstrecken $> 0,3\text{ V} \dots 0,7\text{ V}$ für eine Durchgangsanzeige brauchen, kommt dieses Gerät durch eingebaute Halbleiter nicht zu fehlerhaften Ergebnissen. In vielen Fällen muss kein Bauteil mehr mit einem oder mehreren Anschlüssen für einen Durchgangstest ausgelötet werden. Gerade bei SMD-Platinen ist dies ein wichtiger Nutzen.

Kleine Widerstände exakt messen

Ein weiteres Designziel war die Messung von kleinen Widerständen bis $100\ \Omega$ mit einer Genauigkeit $<5\%$. Im Bereich bis $10\ \Omega$ ist die Messung auf $0,1\ \Omega$ genau. Wichtig ist dies für die Beurteilung des Schutzleiterwiderstandes gem. VDE 0701 (z.B. $<0,3\ \Omega$). Dieser Test liefert einen ersten Messwert. Die bestätigende Messung gem VDE 0701 sollte mit einem Schutzmaßnahmenprüfgerät erfolgen.

Verschiedene Signalcodes für programmierbare Bereiche

In dieser Funktion hat sich der Smarte 100mV-Durchgangsprüfer gut bewährt, weil er ein codiertes Tonsignal liefert. So findet man schnell heraus, ob ein Zuleitungskabel Schwachstellen hat, wenn es bewegt wird, während der Test läuft oder ob z.B. der PE-Anschluss im Geräteinneren mangelhaft ist. Welche Signalcodes in welchem Widerstandsbereich zu hören ist, lässt sich programmieren.

Smarte Kalibrierfunktion gleicht Unzulänglichkeiten des ADC aus

Last-but-not-least ist durch die „smarte Kalibrierfunktion“ die Messung bis $100\ \Omega$ erstaunlich genau, bis $100\ \Omega$ zumindest gut brauchbar.

Die Schaltung

Hardware-Architektur und Messprinzip mit einigen Vorteilen

Kern der Schaltung ist der Spannungsteiler R1 ... R3. Die drei Widerstände sind Metallschichtwiderstände 1% / 0,125 W. Der Eingangskreis ist zudem mit zwei antiparallelen schnellen Schaltdioden BAS20 gegen Überspannungen gesichert. Die Dioden können immerhin 1s lang 0,5A aufnehmen. Wenn man z.B. Elkos testen möchte, die möglicherweise nicht völlig entladen sind, so sichert die Eingangsschaltung dem OpAmp U1 das Überleben.

Der Spannungsteiler ist mit der stabilisierten Betriebsspannung von 3,3V verbunden. Damit durchfließen ihn recht genau 10mA. Sind die Klemmen K1 offen, so stehen hier 100 mV an - unsere Prüfspannung.

Signalverstärkung mit MCP6002 auf 3V

U1 ist mit einem MCP6002 realisiert, der das Eingangssignal auf genau 3,0V verstärken soll. Mit R6 gleicht man die Ausgangsspannung an Pin 1 von U1 auf 3,0V ab. Durch die niedrige Eingangsimpedanz der Schaltung machen sich Brumm oder Einstreuungen nicht bemerkbar. Die Schaltung zeigt auch keinerlei Schwingneigung.

Das Ausgangssignal von U1 gelangt direkt zum ADC des 8266-Prozessors auf dessen einzigem Analogeingang.

PS1240P02BT - der passive Piezobuzzer

Der Buzzer wird von einem BC817 angesteuert, der seinen Basisstrom über R7 von GPIO14 des Prozessors erhält. Erwähnenswert ist hier R8. Der Buzzer gibt nämlich nur dann ein 4 kHz-Piepsen von sich, wenn seine Parallelkapazität über R8 schnell genug entladen wird. Wer bisher mit programmierbaren Buzzern gearbeitet hat, schaut erst einmal nicht ins Datenblatt und übersieht gerne dieses Detail.

Stromversorgung

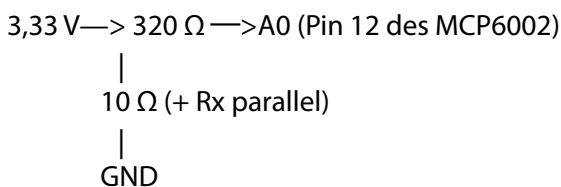
Ein Grund für die Wahl des Heltec WiFi Kit 8 war die integrierte Ladeschaltung für Lipo-Akkus. So angenehm dies auch ist - der Hersteller hat sich entschieden, einen „Micro JSST 2Pin 1,27mm“-Stecker zu verbauen. Dass es Akkus mit dem Gegenstück gibt, konnte ich nicht feststellen. Es gibt jedoch ein Adapterkabel auf 2,0 PH. Und dafür findet man viele Akkus.

Da wir das WiFi-Modul des Boards nicht aktivieren, läuft der Tester ca. 40h mit einem 500mAh-Akku.

Der Sketch

Die „Intelligenz“ der Schaltung liegt im Sketch. Der Sketch misst Widerstände von 0 bis 100 Ω , zeigt den Wert auf einem 128x32 OLED-Display an und gibt akustische Signale über den Piezopiepser aus. Alle Einstellungen werden permanent im Flash-Speicher gespeichert.

Die Schaltung basiert auf einem vorgestellten Spannungsteiler:



Im Ruhezustand (kein Prüfling angeschlossen) fließen durch den Teiler genau 10 mA, und an der Verbindungsstelle liegen 100 mV. Der MCP6002 verstärkt diese Spannung um den Faktor 30, sodass am ADC-Eingang A0 genau 3,00 V anliegen.

Wird ein unbekannter Widerstand Rx parallel zum 10 Ω -Widerstand geschaltet, sinkt der Gesamtwiderstand und damit auch die Spannung am Messpunkt. Aus dieser Spannungsänderung lässt sich Rx berechnen.

Nun gehen wir auf die wichtigsten Funktionen des Sketches ein:

Der ADC des ESP8266

Der ESP8266 hat einen 10-Bit-ADC (Werte 0–1023) mit einer nominellen Referenzspannung von 3,3 V. In der Praxis zeigt dieser ADC jedoch eine ausgeprägte Nichtlinearität — die Kennlinie ist nicht gleichmäßig, sondern im unteren Bereich steiler als im oberen. Eine rein rechnerische Umwandlung der ADC-Werte in den Widerstandswert würde daher systematische Fehler von bis zu 20% erzeugen. Dies ist der Hauptgrund für den Lookup-Tabellen-Ansatz.

Lookup-Tabelle und Interpolation

Statt linear zu rechnen, verwendet der Sketch eine empirisch ermittelte Tabelle, die gemessene ADC-Rohwerte auf an K1 angelegte Widerstandswerte abbildet:

```
float lut_raw[LUT_SIZE] = { 3, 32, 48, 93, ... }; // ADC-Werte  
float lut_rx[LUT_SIZE] = { 0.0, 0.3, 0.5, 1.0, ... }; // Ohm-Werte
```

Dieses Beispiel der Tabelle wurde mit einer Präzisionswiderstandsdekade (Burster) aufgenommen und kompensiert die ADC-Nichtlinearität sowie Toleranzen der Bauteile und den Verstärkungsfehler des Op-Amps.

Stützstellenverteilung

Die 15 Stützstellen der Calibrierfunktion „Full“ sind bewusst ungleichmäßig verteilt:

0 / 0,3 / 0,5 / 1 / 1,5 / 2 / 3 / 5 / 7 / 10 / 15 / 20 / 30 / 50 / 100 Ω

Im Bereich 0–5 Ω ist die Kennlinie steil — kleine Widerstandsänderungen erzeugen große ADC-Wertänderungen. Hier sind die Stützstellen eng gesetzt, um die Interpolation präzise zu halten.

Oberhalb von 20 Ω wird die Kennlinie flacher, weiter auseinanderliegende Stützstellen reichen aus. Der Bereich 0–1 Ω ist für die VDE-0701-Prüfung relevant, die einen Schutzleiterwiderstand $\leq 0,3 \Omega$ zwischen dem PE-Anschluss des Netzkabels und den berührbaren Gehäuseteilen verlangt.

Lineare Interpolation

Zwischen den Stützstellen wird linear interpoliert:

```
float t = (raw - lut_raw[i-1]) / (lut_raw[i] - lut_raw[i-1]);  
return lut_rx[i-1] + t * (lut_rx[i] - lut_rx[i-1]);
```

Der Parameter t läuft von 0,0 bis 1,0 und gibt die relative Position zwischen zwei Stützstellen an. Das Ergebnis ist eine stückweise lineare Approximation der realen Kennlinie.

Division-durch-Null abfangen

```
if (divisor == 0) return lut_rx[i-1];
```

verhindert nan-Werte falls zwei Stützstellen denselben RAW-Wert haben sollten.

Offset-Korrektur

Der ESP8266-ADC zeigt auch bei Kurzschluss der Eingangsklemmen (0 Ω) einen Rohwert von ca. 42, verursacht durch Rauschen, Einstreuungen und den Schleifenwiderstand der Messkabel. Dieser Offset wird vor jeder Berechnung abgezogen:

```
int adc_roh = analogRead(A0) - rawerror;  
if (adc_roh < 0) adc_roh = 0;
```

Das if-Clipping verhindert negative Werte bei sehr kleinen Signalen.

EEPROM-Verwaltung

Der ESP8266 hat kein echtes EEPROM. Die Arduino-Bibliothek emuliert es im Flash-Speicher. Der wichtigste Unterschied zur klassischen EEPROM-Nutzung: Änderungen werden erst durch EEPROM.commit() physisch geschrieben. Fehlt dieser Aufruf, dann fehlen die Daten beim Neustart.

```
EEPROM.begin(EEPROM_SIZE); // Speicherbereich reservieren
EEPROM.put(adresse, wert); // Wert schreiben (nur im RAM-Puffer)
EEPROM.commit();           // Puffer → Flash schreiben
EEPROM.get(adresse, wert); // Wert lesen
```

Speicherlayout

So werden die Daten im EEPROM angeordnet:

Adresse	Inhalt	Größe
0	Magic-Byte	1 Byte
1	Sound ON/OFF	1 Byte
2	Limit 1	4 Byte (float)
6	Limit 2	4 Byte (float)
10	Limit 3	4 Byte (float)
14	Cal-Flag	1 Byte
15	rawerror	2 Byte (int)
17	lut_raw[0..14]	60 Byte (15 × float)

Magic-Byte

Das Magic-Byte (0xAB) an Adresse 0 ist ein Erkennungsmerkmal: beim ersten Start eines neu geflashten Gerätes enthält das EEPROM zufällige Werte. Der Sketch prüft beim Start ob das Magic-Byte korrekt ist:

```
if (EEPROM.read(ADDR_MAGIC) != EEPROM_MAGIC) return; // Defaults behalten
```

Ist es nicht gesetzt, werden die im Sketch hinterlegten Default-Werte verwendet. Nach der ersten gewollten Speicherung wird das Magic-Byte gesetzt und der EEPROM-Inhalt als gültig angenommen.

Sicheres Kalibrieren mit Puffer

Ein kritisches Problem bei der Kalibrierung: würde man jeden gemessenen Stützpunkt sofort ins EEPROM schreiben und der Nutzer bricht mittendrin ab, enthält das EEPROM einen Mix aus alten und neuen Werten — was zu nan-Ergebnissen führt.

Die Lösung ist ein zweistufiger Schreibprozess:

```
float cal_buffer[LUT_SIZE]; // temporärer Arbeitspuffer
```

Während der Kalibrierung schreibt calEinzelMessung() alle Messwerte in cal_buffer[], nicht in lut_raw[]. Erst wenn alle Messungen erfolgreich abgeschlossen sind, überträgt eepromSaveCal() den Puffer:

```
void eepromSaveCal() {
  for (int i = 0; i < LUT_SIZE; i++) {
    lut_raw[i] = cal_buffer[i]; // erst jetzt übernehmen
  }
  // write to EEPROM
}
```

Ein Abbruch hinterlässt so keine Spuren — die alte Kalibrierung bleibt bis zum letzten Moment intakt.

Kalibrierung mit zwei Optionen

Schnellkalibrierung (5 Punkte):

Misst nur 5 Stützstellen (0 / 1 / 10 / 30 / 100 Ω) und interpoliert die restlichen 10 Punkte automatisch:

```
const int QUICK_IDX[] = { 0, 3, 9, 12, 14 };
```

Diese Indizes zeigen auf die entsprechenden Positionen in `lut_rx[]`. Die Interpolation zwischen den gemessenen Punkten erfolgt linear — für manche Anwendungen ist dies ausreichend genau.

Vollkalibrierung (15 Punkte):

Misst alle Stützstellen einzeln. Aufwändiger, aber optimal — kompensiert auch lokale Nichtlinearitäten des jeweiligen ADC-Exemplars.

Offset-Messung

Beide Kalibriermodi beginnen mit einer Kurzschlussmessung (K1 überbrücken) , um den Offset zu ermitteln:

```
rawerror = analogRead(A0);
```

Hier wird der ADC-Rohwert bei Kurzschluss direkt als neuer Offset gespeichert. Dieser Wert ist gerätespezifisch und kann zwischen ESP8266-Exemplaren deutlich variieren.

User Interface mit den 2 Tasten SELECT und ENTER

Beide Tasten sind mit INPUT_PULLUP konfiguriert — sie ziehen den Pin im Ruhezustand auf HIGH. Wird eine Taste gedrückt verbindet sie mit GND und erzeugt LOW. Die Abfrage ist entsprechend invertiert:

```
bool enterPressed() { return digitalRead(PIN_ENTER) == LOW; }
```

Langdruck-Erkennung

Die Funktion `enterHaltezeit()` misst wie lange ENTER gedrückt wird und kann bei Erreichen der Schwelle sofort reagieren:

```
int enterHaltezeit(int schwelle_ms, const char* anzeigeText) {  
    int t = 0;  
    bool signalGegeben = false;  
    while (enterPressed()) {  
        delay(100); t += 100;  
        if (!signalGegeben && t >= schwelle_ms) {  
            beepUI();          // sofortiges akustisches Feedback  
            dispText(anzeigeText, ""); // Display-Rückmeldung  
            signalGegeben = true;  
        }  
    }  
    return t;  
}
```

Der Nutzer bekommt also sofort beim Erreichen der Schwelle (5s für Menü, 3s für Kalibrierung) eine akustische Rückmeldung — er muss nicht raten ob er lange genug gedrückt hat.

Abbruch-Kombination

SELECT + ENTER gleichzeitig für mehr als 1 Sekunde bricht jeden laufenden Prozess ab:

```
bool abbruchGedrueckt() {  
    if (selectPressed() && enterPressed()) {  
        int t = 0;  
        while (selectPressed() && enterPressed()) {  
            delay(100); t += 100;  
        }  
    }  
}
```

```

    if (t >= 1000) { ... return true; }
  }
}
return false;
}

```

Diese Funktion wird in allen Kalibrierungsschleifen regelmäßig aufgerufen, sodass der Nutzer jederzeit aussteigen kann.

Menüdarstellung

Alle Menüs verwenden die einheitliche dispMenu() Funktion:

```

void dispMenu(const char** items, int anzahl, int sel) {
  snprintf(z1, sizeof(z1), "> %s", items[sel]);
  snprintf(z2, sizeof(z2), " %s", items[(sel + 1) % anzahl]);
  dispText(z1, z2);
}

```

Das Display zeigt immer genau zwei Einträge — den aktuell gewählten mit > und den nächsten darunter. Mit SELECT scrollt man durch die Liste, der Modulo-Operator % anzahl sorgt dafür, dass nach dem letzten Eintrag wieder der erste kommt.

Akustische Signale mit dem Piezopiepser

Der Piezopiepser PS1240P02BT ist ein passiver Wandler. Er wird durch die Funktion tone() mit einem Rechtecksignal von ca. 4 kHz angesteuert.

Der BC817-Transistor als Schalter ist sinnvoll, weil der Piepser so mit der LiPo-Spannung (4,2 V) betrieben wird, was einen etwas lautereren Ton ergibt als die 3,3 V der GPIO. Der Wandler verhält sich in etwa wie eine Kapazität von ca. 50 nF. Der Widerstand R8 wird benötigt, um seine Kapazität ausreichend schnell zu entladen. Ansonsten kommt kein Ton zustande.

Codierung des tonsignales

Im Messbetrieb werden vier Impulsmuster verwendet:

Limit 1 ($\leq$ limit1) - default 1 Ω :	Dauerton 4000 Hz	→ Durchgang / sehr niedrig
Limit 2 ($\leq$ limit2) - default 10 Ω :	Kurze Pulse 100ms/100ms	→ mittlerer Widerstand
Limit 3 ($\leq$ limit3) - default 50 Ω :	Mittlere Pulse 100ms/300ms	
> Limit 3 - default >50 Ω :	Langsame Pulse 100ms/700ms	

Display

Der Sketch verwendet zwei Schriftarten mit unterschiedlichen Aufgaben:

ncenB08_tr — 8px hoch, für zweizeilige Menü- und Statusanzeigen
 logisoso24_tf — 24px hoch, für die Messwertanzeige; mit Zentrierung

Alle Messwerte werden horizontal zentriert dargestellt:

```

int breite = u8g2.getUTF8Width(text);
int x = (128 - breite) / 2;

```

Für Messwerte mit "Ohm" wird ein empirischer Offset von -8 Pixeln angewendet, weil der String optisch etwas nach rechts wirkt. "Overflow" wird ohne diesen Offset zentriert.

Praktische Bedienung, Menüführung

Zugang zu den Menüs

Indem man > 5s die ENTER-Taste drückt, gelangt man zum Hauptmenü. Dort navigiert man mit der SELECT-Taste nach unten. Nach dem letzten Eintrag erscheint wieder der erste Eintrag.

Der aktuelle Menüpunkt wird mit einem „>“-Symbol markiert. Drückt man jetzt ENTER, so gelangt man in das Submenü.

Sound ON/OFF

Oberster Menüpunkt. ENTER drücken und es erscheint die Auswahl:

> ON
OFF

Mit der SELECT-Taste die gewünschte Funktion auswählen und ENTER drücken.

Set limits

Nach der Auswahl kann man die Limits setzen, also die Widerstandswerte, ab denen sich die Impulsfolge des Pieptones ändern soll:

> limit 1
limit 2
limit 3

Man legt an die Klemmen K1 einen Widerstand des gewünschten Wertes an und drückt die ENTER-Taste.

Calibration

Nach Auswahl dieser Funktion kann man zwischen den beiden Kalibriermethoden wählen:

> Quick (5 Punkte = Stützstellen)
Full (15 Punkte = Stützstellen); maximale Genauigkeit

Nach Betätigung der ENTER-Taste zeigt das Display, welcher Widerstand nun angeschlossen werden soll. Man legt den Widerstand sorgfältig an und drückt ENTER. Das Gerät speichert den zugehörigen Wert.

Es empfiehlt sich der Einsatz hochwertiger Widerstände bzw. einer entsprechenden Widerstandsdekade.

Reset default

Setzt das Gerät auf die Defaultwerte zurück.

Abbruch

Abbruch eines begonnenen Prozesses: SELECT und ENTER gleichzeitig länger als 1 s drücken und auf den Bestätigungston warten.

Fazit

Der Smarte 100 mV - Durchgangsprüfer hat sich in unserem Repaircafé sowie in der Elektronikentwicklung beim Autor und befreundeten Ingenieuren aufs Beste bewährt und dabei viele Eingriffe in zu testende Platinen vermeiden können. Da das Gerät recht klein ist (und noch weiter verkleinert werden könnte), ist es zu einem ständigen und geschätzten Begleiter geworden.

[1] ELEKTOR 11/12-2025

[2] GitHub-Repository mit Schaltplänen, Platinenlayout und Sketch

[3] art-of-electronics.blog: Community Projekt für Repaircafés, Maker und Elektroniker; Bestellmöglichkeit für Bausätze

[4] Heltec WiFi Kit 8, technische Dokumentation